

SRS v2.1 traceability audit — Majd's Platform

First audited against `Application-Template-SRS_2.md` on 2026-09-24 by reading the code, then kept up to date as each gap was closed (latest update 2026-09-28). "DONE" means the behaviour exists and, for most items, is covered by an automated test; the sections say which. "PARTIAL" says what is still missing. Counts are of the 201 functional requirements in the SRS.

Scorecard (201 functional requirements)

Status	Count
DONE	146
PARTIAL	55
MISSING	0

Group	Total	Done	Partial	Missing
P1 API conventions	8	5	3	0
P2 Modules	8	3	5	0
P3 Repository/UoW	7	5	2	0
P4 Cross-cutting	7	5	2	0
P5 Plugins	42	25	17	0
U1 Skinnable UI	8	4	4	0
U2 Mobile-first	9	5	4	0
U3 App shell	8	8	0	0
F-Authorization	8	7	1	0
F-Users	8	7	1	0
F-Roles	5	5	0	0
F-Account	6	6	0	0
F-Settings	6	5	1	0
F-Audit	6	6	0	0
F-Notifications	9	4	5	0
F-Files	6	3	3	0
F-Localization	6	5	1	0
F-Errors	5	5	0	0
F-Data	6	3	3	0
F-Export	5	4	1	0
F-Dashboard	4	4	0	0
F-Background-Jobs	5	5	0	0
F-Health	4	3	1	0

Group	Total	Done	Partial	Missing
F-APIDocs	4	4	0	0
F-Caching	3	2	1	0
F-Features	4	4	0	0
F-Search	4	4	0	0

P1 — API conventions

- DONE: 001 (GET/POST only), 002, 005 (`ApiControllerBase`), 006 (status-code filter), 007 (model-state + exception middleware).
- PARTIAL: 003 routes mostly follow `/api/{module}/{action}` but `search`, `roles/permissions/*`, `session/permissions`, `permissions/tree` differ. 004 CSV export, file download, profile picture, external login and the Identity API return raw bodies. 008 schema is inferred by Swashbuckle, no `ProducesResponseType`.

P2 — Modules

- DONE: 001 `IFeatureModule`, 005 application parts, 006 EF configs discovered.
- PARTIAL: 002 host still lists module assemblies by hand. 003 Scrutor scan covers service markers, jobs, search; EF configs applied separately. 007 no Contracts projects; Users/Roles reference Authorization directly, Account references Files. 008 removing a module still needs edits in `Program.cs` and the csproj.
- PARTIAL (004): `ModuleOptions.AddModuleOptions<T>(configuration, section)` binds a section through `IOptions<T>` and validates it with data annotations when the application starts, so a wrong value stops the start naming the setting. Applied to `Email:Smtp`, `RateLimiting`, `Docs` and `Resilience` (tested); `Cache` already fails fast; the jobs, metrics and plugin-trust keys are still read directly.

P3 — Repository / Unit of Work

- DONE: 002 `IReadRepository` is a separate no-tracking implementation (`EfReadRepository`), so a query path cannot modify what it reads (tested). 004 `IUnitOfWork` / `EfUnitOfWork` with explicit transactions; repositories in one scope share one context (tested with a rollback). 005 audit stamping, soft delete enforced centrally in `SaveChanges` for every `ISoftDelete` entity (a plain `Remove` only marks it, and a filter hides it), and domain events (`IHasDomainEvents`, `IDomainEvent`): published after the save succeeds, once, and not at all when the save fails (tested; `DiagnosticsPing` raises `PingRecorded`). 006 repositories are registered as open generics, scoped, and injectable directly. 007 `IReadRepository.PagedAsync(request, sortableColumns, selector, spec)` gives a list endpoint paging, sorting and criteria in one call.
- PARTIAL: 001 and 003: `IRepository`, `IReadRepository` and specifications are available for any entity that implements `IEntity<TKey>` (the module entities and the Identity users and roles now do), but only these handlers use them: the Files list, download and delete, the job-run, background-job, audit-log and notification lists, and Diagnostics. The rest still inject `ApplicationDbContext` (users, roles, settings, exports, plugins, search, dashboard) and some need `Include`-style or set-based queries the specification does not express yet.

P4 — Cross-cutting

- DONE: 001, 002 (behavior pipeline: logging, performance, validation, authorization, feature, caching, transaction, audit).

- DONE: 003 markers, and a query opts into caching with `ICacheableQuery` (the enabled-features query does: cached for a minute and dropped the moment a feature is switched; tested).
- PARTIAL: 005 correlation, exception and security-headers middleware; no response-wrapping middleware.
- DONE: 004 decorators (Scrutor `Decorate`: the notification email sender is wrapped by `ResilientEmailMessageSender`, `SmtpEmailSender` is unchanged), 006 Polly (`OutboundResilience`: retry with backoff and jitter only for transient failures, a timeout per attempt that really cancels the SMTP calls, and a circuit breaker; a wrong password is never retried; tuned by the validated `Resilience` section; tested).
- PARTIAL: 007 Mapster: one configuration for the application (`IObjectMapper`, mappings found per module through Mapster `IRegister` scans), used by the Files, Notifications, Jobs and Roles handlers, with a projection form so the mapping still runs in SQL. Users, Audit, Settings, Plugins and Dashboard DTOs are still hand-built.

P5 — Plugins (42)

DONE (25):

- 029 no self-granting (a newly installed plugin's permissions go to nobody but the Administrator role, tested: an ordinary user's permissions contain none of them)
- 030 lifecycle state (each plugin shows Enabled, Disabled, Failed or Uninstalling)
- 036 authenticity (SHA-256 checksum and allow-list as before, plus digital signatures: a publisher signs a digest of the package's contents with an ECDSA P-256 key and adds `plugin.sig`; the platform verifies it against the keys in `Plugins:Trust:Signers`; a signature that does not verify is always refused, `Plugins:Trust:RequireSignature` also refuses unsigned packages; tampering, an untrusted key and a forged key name are tested)
- 040 admin UI (state, permissions, menu, checks, install, enable, disable, uninstall, upgrade, roll back, and a Settings shortcut for a plugin that defines settings)
- 041 diagnostics (each plugin lists its checks: platform version, every dependency, the UI contract, and any load error, with the failing ones opened)
- 014 and 015 pre-built frontend and runtime loading (a manifest `frontend` entry names an ES-module bundle, an optional stylesheet and a contract version; a menu entry with an `element` is drawn by that custom element, loaded only when the user navigates to it; the other entries keep the metadata-driven page; the loader is Web Components, not Native Federation)
- 018 CSS isolation (the element is mounted in a shadow root, so neither side's styles leak; the theme's CSS custom properties pass through, verified live with the Tasks overview)
- 019 compatibility check and graceful failure (the shell refuses a bundle built for another contract or Angular major version and shows an inline message; a bundle that fails to load or does not define its element shows an error with Try again, not a broken shell)
- 016 static assets (`GET /plugins/{id}/{path}` serves the `frontend/` folder of a plugin package, for an enabled plugin only; content types, ETag revalidation so an upgrade is picked up at once, only the allowed kinds of file, path traversal refused, the host's strict Content-Security-Policy on every response; a package may carry those file kinds under `frontend/` and nothing else there)
- 023 plugin localization (a plugin ships `localization/<language>.json`; the entries join the platform's for server messages, notifications and emails, and are served with them so the shell shows the plugin's menu label in the active language; they can add text but never replace a platform translation; a missing or broken file leaves the English text; text inside a plugin's own frontend screens waits for frontend bundles, FR-PLUG-014)
- 013 per-plugin settings (a plugin declares `SettingDefinitions` like any module; they are found by the same scan, appear on the standard settings page under the plugin's group, are stored and cached like any setting, are checked by the plugin's own `validator` when saved, and may only use names starting with the plugin's key so they cannot replace a platform or another plugin's setting)

- 010 dependencies and load order (a manifest `dependencies` list with optional min/max versions; plugins load after what they need; a plugin with a missing, out-of-range, failed or circular dependency is left disabled with the reason and its assembly is not used; a plugin cannot be enabled while a dependency is off, or disabled or uninstalled while an enabled plugin needs it)
- 031 lifecycle hooks (a module may implement `IPluginLifecycle: OnInstall` and `OnUpgrade(fromVersion)` run at the first start that sees the plugin or a different version, `OnEnable` runs before enabling and a failure keeps the plugin off with the error shown, `OnDisable` and `OnUninstall` run after the change and are logged if they fail)
- 007 module + controllers registered
- 008 per-plugin failure isolation with persisted `LastError`
- 009 host-version check (`minHostVersion / maxHostVersion` in the manifest, checked when a package is uploaded and again at load; an incompatible plugin is refused with the reason)
- 021 menu ingested only for enabled plugins
- 025 permissions appear in role editor
- 026 grantable to roles and users
- 027 enforced by the same authorization pipeline
- 028 uninstall cleanup (the plugin's permissions are removed from every role and user that held them and permission caches are refreshed; tested)
- 038 upgrade and rollback (a newer package upgrades, the previous version is kept and can be restored; compatibility is checked)
- 039 every install, upgrade, rollback, uninstall, enable and disable is an audited command gated by `Plugins.Manage`
- 042 the plugin list shows declared permissions with a link to the roles screen (it does not pre-select the plugin's permissions).

PARTIAL (17):

- 001
- 002 (manifest is a small subset)
- 003
- 004
- 005 (startup scan, and staged installs are applied at start)
- 006 (collectible context, never unloaded)
- 011 (prefixed table; the migration lives in the host, a plugin cannot ship its own)
- 012
- 017
- 020
- 022
- 024
- 032 (a package is fully verified, staged and applied at start with an undo if it cannot be applied, so the host is never left half-installed; plugin database migrations are not part of this)
- 033 (API blocked at once; assembly stays loaded)
- 034 (uninstall stops the plugin at once, removes its menu, permissions and registry entry, and deletes its files at the next start; removing its data on explicit confirmation is not offered, data is always kept)
- 035
- 037.

- Not verified live: FR-PLUG-022 route guard on the dynamic route (permission guard is attached in code); FR-PLUG-037 with a throwing plugin.

U1 — Skinnable UI (updated 2026-09-24)

- DONE: 001 no component uses an inline template or inline styles (five were externalized; ESLint now fails the build if one returns, verified with a deliberately bad component). 003 M3 theming. 006 named skins: Azure (default), Ocean, Forest, Sunset, Violet, each one token file under `theme/skins/`, switchable at runtime from the header and persisted (verified: the primary color changes app-wide). 008 lint rules: ESLint (external templates, template accessibility) and Stylelint (no hex, `rgb()` or named colors outside `theme/`); both pass and both were shown to catch violations.
- PARTIAL: 002 tokens now cover spacing, radius, tap target and motion, and the only hard-coded color is the QR background token, but older component styles still use raw pixel values. 004 every component is OnPush (Angular 22 default, and declared), but pages still mix logic and markup. 005 the data grid takes template overrides and the empty/error states accept projected content; not every shared component does. 007 English literals remain in some TypeScript (snackbar messages are translated at display time).

U2 — Mobile-first (updated 2026-09-24)

- DONE: 001 `@container` queries on the data grid and dashboard (verified: at 320px each row becomes a labelled card, no sideways scrolling). 004 44px minimum for icon buttons and buttons (verified: smallest of 30 buttons is 44px). 006 dark mode, reduced motion, and safe-area insets now effective (`viewport-fit=cover` added). 008 responsive drawer.
- DONE: 002 every page (dashboard, users, roles, settings, plugins, audit log, jobs, features, files, exports, account) is tested at 320px wide, in English and in Arabic, and none scrolls sideways (22 browser tests).
- PARTIAL: 003 fluid heading sizes and logical properties in new code; older code still fixed. 005 no native `<dialog>`, lazy images or View Transitions. 007 PWA: web manifest, icons, and an Angular service worker with an offline app shell are built into the production output (`ngsw.json` verified), but service worker registration could not be tested because this embedded browser refuses all service workers, including a trivial test one. 009 lazy routes and budgets (initial 705 kB); no Lighthouse run.

U3 — App shell (updated 2026-09-24)

- DONE: 001, 002 nested menu groups (Administration), 004, 006, 008 plugin menu entries follow enable/disable live without a reload (verified).
- DONE: 003 the user menu has a Settings entry for anyone who may open the settings page.
- DONE: 007 accessibility: skip link, labelled navigation and visible focus, and an automated WCAG 2.1 A/AA scan (axe, including colour contrast) of nine pages and the login page in both the light and the dark theme, all clean (19 browser tests). Automated scans find only part of the problems, so a manual review with a screen reader is still worth doing.
- DONE: 005 the data grid accepts `#loadingTemplate`, `#emptyTemplate` and `#errorTemplate` to replace its loading bar, empty state and error state, and shows a standard error state with Try again when the list failed to load (the users and roles lists use it); tested.

F-Authorization

- DONE: 001, 002, 005, 006 (role-permission and user-role edits both refresh the cache; tested), 007 (last administrator protected on delete, deactivate and role removal; tested).

- DONE: 003 direct user grants and denies (`GET /api/users/permissions/get` explains what the roles give, what was set on the user and the result; `POST /api/users/permissions/update` replaces the set; a deny wins over a role grant, a grant adds to the roles, both take effect at once, and an administrator holds everything anyway; unknown or contradictory permissions are refused; a Permissions dialog on the Users page edits them; tested).
- DONE: 008 (permissions are found by scanning the assemblies, and a disabled plugin's permissions are left out of the role editor and come back when it is enabled; grants already made are kept; tested).
- PARTIAL: 004 done in the behavior, no action filter.

F-Users / F-Roles

- Users DONE: 003, 004, 005, 006, 007 (self-delete, last-admin delete/deactivate and last-admin role removal are all refused; tested). DONE: 008 (`Security.MinPasswordLength`, 6 to 128, enforced by an Identity password validator so it applies to creating a user, registering, changing and resetting; tested). DONE: 001 (the list has an All / Active / Inactive selector, and the export follows it; a browser test covers it). PARTIAL: 002 (password only, no emailed set-password link).
- Roles DONE: 001, 002, 005. DONE: 003 (a new account gets the default role and creating a new default role clears the old one; tested through self-registration), 004 (deleting a role that has users needs the role that takes them over; the delete dialog asks; the users keep access, permission caches are refreshed, and moving to itself or to an unknown role is refused; tested).

F-Account

- DONE: 001, 002, 005 (notification preferences matrix), 006. DONE: 003 (an unknown notification type is a 400, and choosing every channel again removes the stored choice; tested). PARTIAL: (none here), 004 (2FA, language and time zone are stored per user, and every date on screen is shown in the user's time zone through one formatting service).

F-Settings (updated 2026-09-24)

- DONE: 001 definitions declare whether a user may override them (`AllowUserOverride`); 002 precedence User > Application > code default (verified through the API: an application change reached one user but not the user who had overridden it); 003 typed getters plus a permission-guarded Application update and a self-scoped `settings/my` and `settings/update-mine` for the User scope (validates that the setting allows user override and, for time zone, that it exists); 004 caching with invalidation for both scopes.
- DONE: 006 sensitive settings are encrypted at rest (ASP.NET Data Protection, stored as `enc:...`), masked on every read (only `hasValue` is returned), changed write-only (blank keeps, explicit clear removes), and refused at User scope. Verified: database row is ciphertext, the secret appears in no API response, non-admins get 403, and the stored SMTP password is what the mail sender uses.
- PARTIAL: 005 admin page groups by category, now including Email and Appearance; no Notifications group.
- Now used by: the user's language and time zone (Account > My preferences, and the header language switch). Not yet applied: dates are still formatted in the browser's zone, not the saved time zone.

F-Audit (updated 2026-09-24)

- DONE: 001 every audited action records user, action, UTC time, HTTP method and path, duration, client IP, browser, outcome and error (verified). 002 property-level before/after values for each entity an audited action created, changed or deleted, stored in `AuditEntityChanges` / `AuditPropertyChanges` (verified for a role rename and a user delete). 003 paged viewer with text, date-range and outcome filters, a detail endpoint and dialog, and an export that honours the same filters. 004 redaction: credential-like fields (password, hash, token,

stamp, secret, and similar) and encrypted setting values are replaced with [redacted] in both parameters and change values (verified: a created user's password appears nowhere). 005 append-only: there is no modify or delete endpoint (404s verified) and the save interceptor refuses to update or delete audit rows; retention is the only deletion path.

- DONE: 006 refused and failed actions are recorded, including what ASP.NET refuses before a handler runs (no token or an invalid one, a token without the right, a rate-limit refusal), once per caller, address and reason per minute so a scanner cannot flood the trail, and not twice when the pipeline already recorded it; tested. Earlier verification: (verified: a permission denial and a duplicate-role failure appear with their outcomes, and the failure row survives the rolled-back transaction). Denials issued by ASP.NET before the request reaches a handler, such as a missing or invalid token, are not recorded.
- Limits: only `IAuditableCommands` carry data changes; bulk `ExecuteUpdate` / `ExecuteDelete` statements bypass change tracking; notification, job-run and audit tables are excluded on purpose; client IP is the direct connection address (no forwarded-header handling); entries written before this change have no new fields.

F-Notifications (updated 2026-09-24)

- DONE: 005 SignalR hub `/hubs/notifications` pushes to the user in real time (verified live: toast and badge without reload). 006 per-type x channel preferences with an Account matrix, honored by the dispatcher (verified: opted-out type not delivered). 007 email goes through a queue with retry and exponential backoff, then Failed after 3 attempts (verified: rows retried against the rejecting Gmail server).
- PARTIAL: 001 dispatcher is `NotificationPublisher` (same `IUserNotificationPublisher` interface, now typed) rather than a separate `INotificationDispatcher`. 002 two channels (in-app, email); no SMS or push, and no shared `INotificationChannel` interface yet. 004 type stored on the notification; delivery status tracked for queued channels only, no severity or payload. 009 new channel still needs dispatcher edits.
- PARTIAL (008): email provider settings (host, port, user, encrypted password, from) are editable in Settings with a "Send test email" action; SMS/push providers do not exist.
- DONE: 003 templates per channel and type in the recipient's language (`INotificationTemplateRenderer`; the default renderer translates the text and lays email out as HTML whose direction follows the language, with all text encoded; an `INotificationTemplate` registered for a type and channel replaces the default for exactly that pair, and `SecurityEmailTemplate` is the example; the rendered body is stored with the queued delivery; tested).
- Note: real email delivery is blocked by the rejected Gmail credentials; the retry path is proven, a successful send is not.

F-Files

- DONE: 003, 004. PARTIAL: 001 (concrete disk class, no `IFileStorage`, no cloud), 002 (blocked-extension list, size cap, no content sniffing), 005 (streamed, not chunked). DONE: 006 soft delete (deleting sets `DeletedAt` / `DeletedBy`, a global query filter hides the file from every query at once, and `PurgeDeletedFilesJob` removes the bytes and the record after `Files.DeletedRetentionDays`, default 30; the orphan sweep leaves a soft-deleted file's bytes alone; tested).

F-Localization

- DONE: 002 the server answers in the request's language: the client's `Accept-Language` (the Angular app sends the language on screen), else the signed-in user's saved language, else the application default; error and validation messages (the platform's own and `FluentValidation`'s built-in translations), notification texts (written per recipient) and the account emails (right to left for Arabic) are translated, `GET /api/localization/languages` and `GET /api/localization/resources?culture=` exist. 003 language switch at

runtime without a reload, per-language file loaded on demand (tested). 004 dates in the user's time zone (`Appearance.Timezone`), numbers and currency (`Appearance.Currency`, a new personal setting) formatted by the active language through `localDate`, `localNumber` and `localCurrency`; server timestamps are read as UTC. 005 default language (`General.DefaultLanguage`) and fallback: a missing translation or an unsupported language shows the English source, never a blank (tested at both ends). 006 the direction flips with the language (tested), and lint now rejects physical `left / right` CSS so new styles stay right-to-left safe; no page-by-page visual audit in a browser was done.

- PARTIAL: 001 all of the interface's own text and every message the server writes is keyed and translated, and tests fail when one is missing (a scan of the templates for the frontend, a scan of the source for the backend). Identity's own texts (password rules, taken names) are translated too, several joined in one message one by one. Not covered: model-binding errors, text inside plugin frontend screens (plugins can translate their menu labels and server messages, P5 FR-PLUG-023), and Swagger.

F-Errors

- DONE: 001, 002, 004 (a central interceptor handles 401, 403, 429, unreachable server and 5xx with a translated message, and leaves field errors to each screen). DONE: 003, 005 Serilog is the application's logger (`Logging:Level`, `Logging:Console`, and a daily rolling file under `Logging:File:Path` keeping `Logging:File:RetainedFiles` files, all checked at start; every line carries the request's correlation id; tested).

F-Data

- DONE: 002, 003 (DB-side paging, tied to the repository through `PagedAsync`; a sort column outside the list's allow-list is a 400 naming the allowed ones, not a silently unsorted list; tested), 004 (`Paging:MaxPageSize`, default 100, checked at start). PARTIAL: 001 column filters are still written per list, 005 grid has no filter UI, 006 permission checks live in consumers.

F-Export / F-Dashboard / F-Background-Jobs

- Export: DONE 001 (CSV, Excel and PDF from one renderer, `TabularExport`, so all formats hold the same rows; users and audit exports take the list's filters and share its filter code), 002 (server-generated PDF, landscape A4, paginated, capped at 2,000 rows and says so). Formats: `?format=csv|xlsx|pdf`; unknown values are a 400. DONE 003 (import from CSV or Excel: per-row validation through the same `CreateUserCommand` as the create endpoint, valid rows imported, invalid rows reported with row number and reason, whole-file problems such as a missing column rejected up front; users import is wired end to end with a reusable dialog), 005 (downloadable CSV and Excel templates, headers only, plus an Instructions sheet). PARTIAL 004: exports run as background jobs (`ExportJob` queue, hosted worker, atomic claim, restart recovery, retention cleanup), the file is delivered through F-Files and the user gets a notification that links to My exports (AC-EXP-3); imports still run during the request (5,000-row limit), so the requirement is not fully met. Only users have an import so far; other resources reuse `TabularReader`, `ImportRunner` and `ImportTemplate`.
- Dashboard: DONE 001-004. Widgets implement `IDashboardWidget` and register from any module; the dashboard lists those the caller may see and each tile loads its own data (deferred until in view). Baseline widgets: users KPI, unread-notifications KPI, failed-actions KPI, 7-day activity chart, recent-activity feed. Layout (order, hidden) is a per-user setting, `Dashboard.Layout`. Tests: `DashboardTests`, `DashboardHandlerTests`, `dashboard.spec.ts`.

Jobs: DONE 001 (recurring jobs by interval or cron expression, plus persisted fire-and-forget and delayed jobs through `IBackgroundJobQueue`)

- 002 (queued jobs are stored in `BackgroundJobs` and survive a restart; a failure is retried with 10 s, 20 s, 40 s ... up to an hour, then left failed and the administrators are told; a failed recurring run is retried after 1, 2 and 4

minutes; a job left running by a restart is run again, and one that keeps crashing the process still ends up failed; tested)

- 003 (queue view with status filter, retry and delete, gated by `Jobs.View` / `Jobs.Manage`; recurring jobs show their last result, failure streak and next run, and can be run now)
- 004 (the queuing user's id and name and a parameter map are stored with the job and handed to it)
- 005 (audit retention, notification cleanup, export cleanup, orphaned uploaded files and abandoned plugin staging folders). Not done: no built-in feature uses the queue yet (exports and email delivery keep their own persisted queues), and a single node is assumed for the recurring scheduler.

F-Health / F-APIDocs / F-Caching

- Health: DONE 001 (readiness checks the database, file storage, cache and email; a critical one down is a 503, a non-critical one is Degraded), 002 (`/metrics` in the Prometheus format: request rate, latency and errors by status code, guarded by `Metrics:Token`), 004 (readiness goes unhealthy when a critical dependency is down; tested). PARTIAL 003 (the correlation id is the W3C trace id, an incoming `traceparent` is continued, the id is carried as baggage and on every log line; spans are not exported to a tracing backend).

ApiDocs: DONE 001 (an OpenAPI document generated from the controllers, described as the `ResponseDto` envelope; a new endpoint appears by itself)

- 002 (a bearer scheme applied to every operation, so the UI's Authorize sends the token)
- 003 (API versioning by query or header with 1.0 as the default, one document per version; only 1.0 exists and a 2.0 controller in the test project proves a new version documents itself)
- 004 (off outside Development unless `Docs:Enabled`; then `Docs:Access` = `Permission` (`Docs.View`), `Authenticated` or `Open`). Not done: generating the Angular client from the document.
- Caching: DONE 001 (`ICacheService`: get, set with a time to live, remove, get-or-add), 003 (permissions, settings, feature flags and cacheable queries use it and are invalidated on write; tested on the serializing path). PARTIAL 002 (Redis is selected by `Cache:Provider=Redis` and its wiring is tested, and a shared distributed cache is proven with two simulated servers, but nothing has been run against a live Redis server, so AC-CACHE-1 is not verified end to end).

F-Features / F-Search

- Features: all 4 DONE (per-user override is optional in the spec and absent).
- Search: all 4 DONE (providers, permission filtering, debounced grouped UI).

Non-functional requirements (updated 2026-09-24)

- DONE: NFR-SEC-1 deny-by-default (fallback authorization policy; verified anonymous 401 on unmarked endpoints, Identity `/manage/*` still protected), NFR-SEC-4 rate limiting (auth endpoints 10/min per IP, exports and search 30/min per user, global 600/min; verified 429 with the standard envelope and `Retry-After`), NFR-SEC-6 no stack traces to clients, NFR-MAINT-1 modular projects, NFR-OBS-1 correlation and user id in logs.
- PARTIAL: NFR-SEC-2 HTTPS redirect, config-driven CORS, HSTS (non-development only) and security headers (nosniff, frame deny, referrer policy, permissions policy, COOP, strict CSP; verified on responses). Remaining: the Angular host serves its own headers, which are not configured; HTTPS-only is not enforced in development. NFR-SEC-3 (sort allow-list only partly checked), SEC-5 (settings secrets encrypted at rest; SMTP config secrets still live in user-secrets), PERF-1..4 (nothing measured), SCALE-1 (SQLite and in-memory caches; limiter counters are per instance), OBS-2 (health only), A11Y-1, PRIV-1, PRIV-2.
- MISSING: NFR-SCALE-2 Redis backplane for SignalR (the hub itself exists; the cache can already use Redis), NFR-MAINT-3 .NET analyzers (CI and the frontend linters exist).

Definition of Done — tests (updated 2026-09-25)

- **Backend: 428 automated tests** in `src/MajdsApp.Tests` (343) and `src/MajdsApp.Tests.Plugins` (84, a separate process because EF caches its model per process and a plugin's entity can only be in it once), run with `dotnet test MajdsApp.slnx` (under a minute). A GitHub Actions workflow (`.github/workflows/ci.yml`) runs them, and the frontend lint, tests and production build, on every push and pull request.
- **Covered end to end:** deny-by-default and hardening headers, login, rate limiting, roles/users/permissions (with permission-denied and validation-failure cases), settings scopes and encrypted secrets, the audit trail, notifications and their per-recipient language, forgot/reset password and registration, dashboard widgets and layouts, exports in three formats and imports with per-row errors, background exports and the persisted job queue (retries, backoff, restart recovery), plugin install/upgrade/rollback/uninstall with the trust policy, localization, the cache backends (including the whole app on the serializing path), health and metrics, correlation ids, and the API docs and versioning.
- **Frontend: 127 automated tests** (vite via `ng test`): the shell, menu and theme, interceptors (auth, error, language), the login page, the data grid and shared components, the dashboard, export menu and import dialog, the exports, jobs and plugins screens, language switching and formatting, and a translation-coverage scan that fails when text has no Arabic entry. ESLint and Stylelint (which now rejects physical left/right CSS) run in CI.
- **Bugs the tests found and that are now fixed:** a second host in the same process took over the first one's log output (each host now keeps its own Serilog logger); the last administrator could lose the Administrator role by editing the user (FR-USER-007); a user's cached permissions were not refreshed after a role change (FR-AUTHZ-006); a hand-placed folder in the plugin staging area would have been applied at startup without verification; error responses were counted as 200 in the metrics because the metrics middleware sat inside the exception handler; and the API never bound `Email:Smtp` to its options, which is why every email was rejected.
- **Browser end-to-end tests: 49** (8 journeys, 19 accessibility scans, 22 narrow-screen checks) (Playwright driving the installed Chrome against the real API and a throwaway database, `npm run e2e`, also a CI job). The journeys: wrong and right sign-in, a protected page sending a visitor to the login page, the Arabic right-to-left layout with the menu on the right and its persistence after a reload, the phone-width header, a plugin's own screen in a shadow root, and deleting a role that has users through the reassignment dialog. The scans found one real problem (the notification count was on an icon without a text alternative, on every page), now fixed by moving the badge to the button with a spoken description.
- **Also fixed after looking at the running site:** the fonts and icons came from Google's servers, so the whole interface lost its icons without internet; Roboto and the Material icons are now served with the app (which also makes the offline shell complete). Dashboard tiles now reload when the language changes, because their captions and day names are worded by the server.
- **Not covered:** the two-factor and Google sign-in flows, the file upload and download endpoints over HTTP, SignalR delivery, accessibility and Lighthouse checks, Testcontainers/SQL Server (tests use SQLite, the same provider the app uses today), and a live Redis server.
- **Documentation:** every project has a README, including one for each module; each module README lists its endpoints, permissions, settings, tables, jobs and configuration keys.

Outside the SRS

- Email works end to end (registration confirmation, forgot-password, admin reset, notification email). The 535 rejection was not a bad password: the API never read `Email:Smtp` into its options.
- The SRS's P1 note says `Errors` stays `[JsonIgnore]`; this project deliberately serializes it, which the same note lists as an allowed revision, so validation messages reach the UI.

Order the gaps were closed, and what is left

Done (2026-09-24): notifications with SignalR and preferences; user-scope and encrypted settings; security hardening; audit trail; frontend rules, skins, PWA build and container queries; automated tests; READMEs; CI; working email.

Done (2026-09-25): dashboard widgets; export to Excel and PDF, import with templates, background exports; plugin install, upgrade, rollback and uninstall with a trust policy; the persisted job queue with cron, retries and monitoring; localization (server messages, notifications, emails, formatting, right-to-left guard); the cache abstraction with a Redis option; health checks, metrics and trace propagation; API docs, versioning and a production gate.

Still open, roughly by value (no requirement is missing and 55 are partial):

1. P5: plugins shipping their own database migrations, applying changes without a restart, and native federation (only Web Components are supported).
2. P4: Mapster for the remaining DTOs; P2: options classes for the remaining keys; P3: moving the remaining handlers to the repository (most partial items are here and in P5).
3. F-Notifications SMS and push channels and a shared channel interface; F-Files storage interface.
4. Verification: more browser end-to-end journeys (seven exist), a run against a live Redis, a Redis backplane for SignalR, exporting traces (OpenTelemetry), generating the Angular client from the OpenAPI document, background imports.